CUDA Acceleration and Empirical Stopping-Depth Regimes in Iterative Max–Min Forgotten Effects

Carlos Riquelme-Ávila*, Julio Rojas-Mora†, and Elliott Mardones-Arias‡
*School of Computer Science, Universidad Católica de Temuco, Temuco, Chile
criquelme2022@alu.uct.cl
†Department of Computer Science, Universidad Católica de Temuco, Temuco, Chile
julio.rojas@uct.cl
‡PhD Program in Territorial Planning and Sustainability, Universidad Católica de Temuco, Temuco, Chile
elliott.mardones@uct.cl

***Abstract*—Forgotten-effects computation requires repeated max–min convolutions over a weighted influence matrix, with cubic arithmetic cost per propagation order and potentially cubic intermediate memory in tensor-broadcasting implementations. This paper presents forgethreads, a CUDA implementation that evaluates each max–min update directly at the cell level and stores only quadratic-size matrices. On dense benchmark instances, forgethreads obtains speedups between $12.8\times$ and $44\times$ against a TensorFlow-based implementation and extends the largest completed dense benchmark size from $1{,}200$ to $32{,}000$ nodes on an NVIDIA RTX 4090. The stopping-depth experiment uses a binary one-way block Bernoulli design with 2640 observations. The main empirical result is a size-dependent finite-range two-window profile: at larger sampled graph sizes, high stopping depths appear near two block-level effective-degree windows, reached near $c = 1$ and $c = 2$ in the balanced design, with an inter-threshold valley between them and a shortcut-dominated decline at larger c. Within the sampled finite design, the observed stopping-depth profile is consistent with organization by block-level effective degrees rather than by global connectivity alone.**

***Index Terms*—forgotten effects, max–min convolution, CUDA, stopping depth, random block structure**

I. Introduction

The forgotten effects method, introduced by Kaufmann and Gil-Aluja [1], studies indirect influences that are not visible in the direct incidence matrix of a fuzzy relational system. Given a weighted influence matrix $E \in [0,1]^{N\times N}$, the method repeatedly applies a max–min convolution to obtain cumulative indirect effects between ordered pairs of nodes and tracks the increase in incidence produced at each iteration, allowing significant indirect effects that emerge through progressively longer paths to be identified. The usual computational question is to obtain these indirect effects efficiently; the empirical question studied here is different: how many propagation orders are needed before no new significant effect remains. The answer matters in two ways. Computationally, η_0 determines how many max–min propagation steps must be executed, and therefore directly scales the total computational cost. Methodologically, η_0 is the last order at which a significant forgotten effect can still appear, so it bounds the observed indirectness of significant effects in an application of the method.

979-8-3195-1296-3/26/$31.00 ©2026 IEEE

The computational cost of this procedure is high. A direct max–min convolution evaluates N^2 output cells, each requiring a reduction over N intermediate nodes. Thus, the arithmetic cost per order is $\mathcal{O}(N^3)$. Although the input contains a single $N \times N$ influence matrix and therefore requires only $\mathcal{O}(N^2)$ storage, the TensorFlow max–min convolution broadcasts the matrix operands over the intermediate index k. This operation materializes candidate values for all triples (i,k,j), yielding $\mathcal{O}(N^3)$ intermediate memory even in the single-matrix case. This materialization pattern limits graph size even on high-end GPUs.

We address the memory limitation by developing and evaluating **forgethreads**, a CUDA implementation for forgotten-effects computation. The recurrence and its $\mathcal{O}(N^3)$ arithmetic cost per propagation order are unchanged. The implementation changes the execution model: each max–min update is evaluated directly at the output-cell level, so the computation stores only quadratic-size matrices instead of materializing the cubic tensor of intermediate values.

The contribution of this paper is twofold but operationally connected. The computational contribution is a CUDA implementation that evaluates max–min propagation with quadratic working memory and extends the completed dense range relative to the TensorFlow reference. The empirical contribution uses the same iterative propagation model to study η_0, defined as the last propagation order with $q_r > 0$, across 2640 binary one-way block Bernoulli instances. Within the sampled design, the experiment identifies a finite-range two-window profile, with high stopping depths near the induced block-level effective-degree windows and shorter depths in the shortcut-dominated branch.

Forgotten-effects methodology supplies the application framework [1], with recent work illustrating higher-order influences in complex networks [2]. Max–min composition and its algebraic foundations are developed in fuzzy-relation theory [3]–[6]. CUDA provides the execution model [7], while GPU path algorithms have also used matrix and semiring formulations [8], [9]. These works target path or closure values rather than the order-wise significant-increment stopping

statistic considered here. Li et al. [10] analyze entrywise max–min closure values in random Bernoulli fuzzy matrices; our observable is instead the global stopping depth η_0.

The paper is organized as follows. Section II defines the matrix structure and max–min primitives. Section III presents the CUDA implementation and the principal package algorithms. Section IV reports runtime and memory benchmarks. Section V defines the stopping-depth experiment. Section VI presents the empirical regime analysis. Section VII discusses the computational and empirical implications, limitations, and future work, and Section VIII concludes.

II. Matrix Structure and Max–Min Primitives

A. Random Block Structure

Let $\mathcal{V} = \mathcal{N} \cup \mathcal{M}$ be a node set partitioned into two blocks, with sizes $|\mathcal{N}| = n_{\mathcal{N}}$, $|\mathcal{M}| = n_{\mathcal{M}}$, and $N_{\mathrm{tot}} = n_{\mathcal{N}} + n_{\mathcal{M}}$. The weighted influence matrix is $E \in [0,1]^{N_{\mathrm{tot}} \times N_{\mathrm{tot}}}$, and reflexivity is imposed by $E_{ii} = 1$ for all $i \in \mathcal{V}$. We use the one-way block structure

$$E = \begin{pmatrix} E_{\mathcal{N}\mathcal{N}} & E_{\mathcal{N}\mathcal{M}} \\ 0 & E_{\mathcal{M}\mathcal{M}} \end{pmatrix},$$

where the lower-left block suppresses feedback from $\mathcal{M}$ to $\mathcal{N}$. Thus, the only admissible inter-block direction is $\mathcal{N} \to \mathcal{M}$.

In the stopping-depth experiments, the admissible off-diagonal entries in $E_{\mathcal{N}\mathcal{N}}$, $E_{\mathcal{N}\mathcal{M}}$, and $E_{\mathcal{M}\mathcal{M}}$ are sampled according to the Bernoulli design specified in Section V-B. Cycles are allowed inside $E_{\mathcal{N}\mathcal{N}}$ and $E_{\mathcal{M}\mathcal{M}}$. The diagonal entries remain fixed at one and serve as the identity device that makes the max–min iteration cumulative.

B. Max–Min Cumulative Matrices

The base matrix is $E^{(1)} := E$. For $r \geq 1$, cumulative max–min propagation is

$$E_{ij}^{(r+1)} = \max_{k \in \mathcal{V}} \min\left(E_{ik}^{(r)}, E_{kj}\right). \tag{1}$$

Equation (1) is the max–min convolution kernel used to generate the cumulative matrices. The matrix $E^{(r)}$ stores cumulative influence up to order r, because the reflexive entries $E_{ii} = 1$ preserve previously reached values. Forgotten effects are extracted only after this cumulative stage, by comparing consecutive cumulative matrices.

C. Increment Matrices and Significant Effects

Forgotten effects are identified from increments between consecutive cumulative matrices. We write

$$\Delta^{(1)} := E^{(1)}, \qquad \Delta^{(r)} := E^{(r)} - E^{(r-1)} \quad (r \geq 2),$$

with ordinary arithmetic subtraction. Given $\epsilon \in (0,1)$, the ordered pair (i,j) has a significant increment at order r when

$$\Delta_{ij}^{(r)} > \epsilon.$$

For $r \geq 2$, such an increment is a significant indirect effect from i to j at order r.

A directed path of length ℓ from i to j is a sequence $\gamma = (i_0, \ldots, i_\ell)$ with $i_0 = i$, $i_\ell = j$, and $E_{i_{s-1} i_s} > 0$ for $1 \leq s \leq \ell$.

For each order r, define the significant-increment count

$$q_r := \left|\{(i,j) \in \mathcal{V} \times \mathcal{V} : \Delta_{ij}^{(r)} > \epsilon\}\right|.$$

The deterministic stopping criterion is

$$q_r = 0.$$

The pairwise stopping level is

$$\eta_{ij} := \max\{r \geq 1 : \Delta_{ij}^{(r)} > \epsilon\},$$

with $\eta_{ij} = 0$ if no significant increment occurs. The global stopping depth is

$$\eta_0 := \max_{i,j \in \mathcal{V}} \eta_{ij}.$$

Equivalently, η_0 is the last order at which $q_r > 0$. The empirical sections estimate how this stopping depth varies with the sampled matrix family.

For the threshold used in the stopping-depth experiment, $\epsilon = 0.5$, the pairwise stopping level has a uniqueness interpretation. More generally, if $\epsilon \geq 1/2$, monotonicity and boundedness imply that each ordered pair can have at most one significant increment. Indeed, two distinct orders $r < s$ with

$$\Delta_{ij}^{(r)} > \epsilon \quad \text{and} \quad \Delta_{ij}^{(s)} > \epsilon$$

would force the cumulative entry $E_{ij}^{(s)}$ to increase by more than $2\epsilon \geq 1$, contradicting $E_{ij}^{(s)} \leq 1$. Thus, at $\epsilon = 0.5$, η_{ij} is the unique significant-increment order of the pair, and η_0 is the deterministic stopping depth of the whole propagation.

Consequently, significant increments form a finite resource: each ordered pair contributes at most once, so

$$\sum_{r \geq 1} q_r \leq N_{\mathrm{tot}}^2.$$

This finite-resource property motivates the stopping criterion implemented in Section III.

III. CUDA Implementation

A. Cell-Level Convolution Kernel

The CUDA kernel receives the current cumulative matrix $E^{(r)}$ and the base matrix E, and writes the next cumulative matrix $E^{(r+1)}$. It evaluates (1) directly at the cell level: each CUDA block is assigned one output cell (i,j). Threads inside the block scan subsets of the intermediate index k, compute local candidates $\min(E_{ik}^{(r)}, E_{kj})$, and reduce them in shared memory to write $E_{ij}^{(r+1)}$.

The kernel also stores

$$\kappa_{ij} \in \operatorname*{argmax}_k \min(E_{ik}^{(r)}, E_{kj}),$$

which can be used for path reconstruction. After the output cell is computed, the corresponding increment $\Delta_{ij}^{(r+1)}$ is tested against ϵ. If $\Delta_{ij}^{(r+1)} > \epsilon$, an atomic counter is increased.

Algorithm 1 Cell-level CUDA kernel for one max–min propagation step.

Require: Current cumulative matrix $E^{(r)}$, base matrix E, threshold ϵ
Ensure: Next cumulative matrix $E^{(r+1)}$, argmax buffer $\kappa^{(r+1)}$, significant-increment counter q_{r+1}

Launch one CUDA block for each output cell (i, j)
for all cells (i, j) in parallel **do**
  $v_{max} \leftarrow -\infty$
  $v_{argmax} \leftarrow 0$
  for all blocks t of size T covering $\mathcal{V}$ **do**
    Cooperatively transfer the entries for $k \in t$ to shared memory
    $u \leftarrow \max_{k \in t} \min(E^{(r)}_{ik}, E_{kj})$
    $k^\star \in \operatorname{argmax}_{k \in t} \min(E^{(r)}_{ik}, E_{kj})$
    if $u > v_{max}$ **then**
      $v_{max} \leftarrow u$
      $v_{argmax} \leftarrow k^\star$
    end if
  end for
  $E^{(r+1)}_{ij} \leftarrow v_{max}$
  $\kappa^{(r+1)}_{ij} \leftarrow v_{argmax}$
  $\Delta^{(r+1)}_{ij} \leftarrow E^{(r+1)}_{ij} - E^{(r)}_{ij}$
  if $\Delta^{(r+1)}_{ij} > \epsilon$ **then**
    Atomically increment q_{r+1}
  end if
end for
return $E^{(r+1)}, \kappa^{(r+1)}, q_{r+1}$

B. Python Interface

The implementation is written in CUDA C++ and exposed through a Python interface using PyBind11 [11]. It accepts GPU-resident tensors and NumPy-compatible arrays. DLPack is used when possible to avoid unnecessary host–device copies. The public routine returns indices, values, and a stopping depth. The software routine can be evaluated at arbitrary thresholds in $(0, 1)$. The stopping-depth interpretation used in this paper is the $\epsilon = 0.5$ case, for which the uniqueness argument in Section II-C applies.

C. Core Algorithms

The package is organized around the single-step CUDA primitive CUDAMAXMINSTEP, specified in Algorithm 1. Given the current cumulative matrix $E^{(r)}$, the base matrix E, and a threshold ϵ, this primitive computes the next cumulative matrix $E^{(r+1)}$, stores an argmax buffer $\kappa^{(r+1)}$ for optional path reconstruction, and returns a counter q_{r+1} equal to the number of entries whose new increment exceeds ϵ.

Algorithm 1 is written at the logical cell level. In the implementation, a cooperative block-level load into shared memory is performed, storing the elements of $E^{(r)}$ and E needed for a partial max–min computation. This procedure is repeated until the entire dimension $\mathcal{V}$ is covered. Only the resulting cell value, the argmax index, and the increment flag are written to global memory.

Algorithm 2 Iterative CUDA max–min propagation.

Require: Matrix $E \in [0, 1]^{N \times N}$, threshold ϵ
Ensure: Global stopping depth η_0

$E^{(1)} \leftarrow E$
$\eta_0 \leftarrow 1$
$r \leftarrow 1$
$q \leftarrow 1$
while $q > 0$ **do**
  $(E^{(r+1)}, \kappa^{(r+1)}, q) \leftarrow$ CUDAMAXMINSTEP$(E^{(r)}, E, \epsilon)$
  if $q > 0$ **then**
    $\eta_0 \leftarrow r + 1$
  end if
  $r \leftarrow r + 1$
end while
return η_0

Algorithm 2 wraps repeated calls to CUDAMAXMINSTEP in the outer loop for the stopping criterion defined in Section II-C. The routine initializes the stopping depth at 1, because $E^{(1)} = E$ is the base cumulative matrix, and repeats max–min propagation until the thresholded increment matrix becomes empty. Optional path reconstruction uses the stored argmax buffer in validation workflows, but it is not part of the stopping-depth sweep.

The stopping condition $q_r = 0$ means that the thresholded increment matrix at the last computed order is empty: no entry satisfies $\Delta^{(r)}_{ij} > \epsilon$. The routine therefore returns the last order at which at least one significant increment is observed. This is the computational stopping depth η_0 defined in Section II-C.

Termination does not require an externally prescribed maximum order. The cumulative matrices are monotone nondecreasing and bounded above by one. Moreover, by induction on (1), each entry $E^{(r)}_{ij}$ equals the largest value of $\min_{1 \le s \le \ell} E_{i_{s-1} i_s}$ over directed paths of length $\ell \le r$ from i to j. Removing a cycle from a path cannot decrease this minimum, so every attainable value is already attained by a simple directed path, of length at most $N_{\mathrm{tot}} - 1$. Hence no entry can increase after order $N_{\mathrm{tot}} - 1$, and the recurrence reaches closure after at most $N_{\mathrm{tot}} - 1$ nontrivial propagation orders. The algorithm may compute one additional zero-increment step to verify that the thresholded increment matrix is empty.

IV. Runtime and Memory Benchmarks

A. Benchmark Design

The first experimental block evaluates raw computational performance against **forgeffects**, a TensorFlow-based implementation [12], [13] of forgotten-effects propagation. The benchmark uses dense binary block-structured matrices and computes forgotten effects up to order three. Path assembly is excluded from both implementations. The **forgethreads** implementation evaluated here corresponds to the CUDA

TABLE I
RUNTIME COMPARISON FOR DENSE MAX–MIN FORGOTTEN-EFFECTS COMPUTATION. TIMES ARE REPORTED IN MILLISECONDS.

Nodes	forgethreads	forgeffects	Speedup
150	0.183	8.050	44.02x
300	0.226	8.885	39.31x
600	0.753	15.578	20.67x
700	1.019	19.778	19.40x
900	1.874	40.612	21.68x
1200	4.465	57.175	12.80x

max–min propagation path of the feCUDA code base [14]; the repository reference identifies the software lineage, while the present paper reports the CUDA kernel and stopping-depth experiments. All measurements were obtained on an NVIDIA RTX 4090 with 24 GB of VRAM, and each runtime is reported in milliseconds. Each reported runtime is the mean of ten measured executions following one warm-up execution. GPU completion is forced for both implementations before the end of each timing interval, using the synchronization mechanism appropriate to each implementation. The stopping-depth sweep uses the same termination criterion as Algorithm 2: propagation stops when the next propagation order produces no entries with increment greater than ϵ. Both implementations received the same binary matrix instances and used half-precision (`float16`) representations for the benchmarked max–min propagation. The benchmark configuration, random-seed policy, software environment, and code/data availability information are reported with the released artifact.

B. Runtime Results

Table I shows that **forgethreads** is faster for all sizes completed by both implementations. Speedups range from $44.0\times$ at 150 nodes to $12.8\times$ at 1200 nodes. The speedup shows an overall downward trend with increasing N_{tot}, although the decrease is not monotonic across the tested sizes.

At every tested size from $N_{\mathrm{tot}} = 1{,}300$ through the largest benchmark case, $N_{\mathrm{tot}} = 32{,}000$, **forgeffects** fails to complete due to GPU memory exhaustion, whereas **forgethreads** completes all cases.

C. Memory Model and Scalability

Since **forgeffects** implements (1) through TensorFlow broadcasting, it materializes candidate values for all triples (i, k, j). Although the input matrix itself requires only $\mathcal{O}(N_{\mathrm{tot}}^2)$ storage, this candidate tensor has $\mathcal{O}(N_{\mathrm{tot}}^3)$ memory complexity. The reference workflow also creates additional intermediate tensors during subsequent reduction and index-extraction operations. The exact composition of peak GPU memory depends on TensorFlow's execution and allocation behavior and is not profiled here. Empirically, **forgeffects** exhausts the available 24 GB of GPU memory from $N_{\mathrm{tot}} = 1300$ onward at the tested sizes.

The CUDA kernel used in **forgethreads** stores the base matrix E, the current cumulative matrix $E^{(r)}$, the next cumulative matrix $E^{(r+1)}$, the argmax buffer, and a counter

TABLE II
OBSERVED COMPLETION AND OUT-OF-MEMORY OUTCOMES ON RTX 4090 AT THE TESTED NODE SIZES.

Implementation	Nodes	Result
forgeffects	1,200	Completed
forgeffects	1,300	Out of memory
forgethreads	1,300–32,000	Completed at all tested sizes

for significant increments. Its main memory requirement is therefore $\mathcal{O}(N_{\mathrm{tot}}^2)$. Table II reports the observed completion and out-of-memory outcomes in the RTX 4090 memory-scaling check.

This extended feasible range shows that forgotten-effects propagation remains practical up to the largest tested size on the GPU used for benchmarking; the binary stopping-depth experiment complements the benchmark by exercising the same stopping criterion on a controlled one-way Bernoulli family.

V. EMPIRICAL STOPPING-DEPTH EXPERIMENT

A. Experimental Matrix Families

The two experimental blocks use binary block-structured inputs for distinct purposes. The computational benchmark evaluates runtime and memory behavior, whereas the stopping-depth experiment uses one-way block Bernoulli matrices to study the empirical behavior of η_0. Binarity removes weight-amplitude variation from the stopping experiment. Since the cumulative matrices are monotone, each entry can only remain unchanged or increase from 0 to 1; therefore, every nonzero increment equals 1 and exceeds $\epsilon = 0.5$. By Section II-C, each ordered pair can have at most one such significant increment. The stopping-depth workflow records η_0 together with the order-level significant-increment counts q_r.

B. One-Way Block Bernoulli Design

For the stopping-depth experiment, the one-way block structure of Section II-A is specialized to binary Bernoulli matrices $E \in \{0,1\}^{n\times n}$. The node set is partitioned into $\mathcal{N} \cup \mathcal{M}$, with $n_{\mathcal{N}} = \lfloor \alpha n \rfloor$ and $n_{\mathcal{M}} = n - n_{\mathcal{N}}$; the main experiment uses $\alpha = 0.5$. Diagonal entries are fixed at one.

Admissible off-diagonal entries are sampled independently. A row in $\mathcal{N}$ has $n_{\mathcal{N}} - 1$ admissible targets inside $\mathcal{N}$ and $n_{\mathcal{M}}$ admissible targets in $\mathcal{M}$. These entries are sampled with

$$p_N = \frac{c}{n-1}.$$

A row in $\mathcal{M}$ has no admissible targets in $\mathcal{N}$ and $n_{\mathcal{M}} - 1$ admissible targets inside $\mathcal{M}$. These entries are sampled with

$$p_M = \frac{c}{n_{\mathcal{M}} - 1}.$$

By linearity of expectation, the expected block-level out-degrees are

$$\begin{aligned} d_{\mathcal{N}\mathcal{N}} &= (n_{\mathcal{N}} - 1)p_N, \\ d_{\mathcal{N}\mathcal{M}} &= n_{\mathcal{M}} p_N, \\ d_{\mathcal{M}\mathcal{M}} &= (n_{\mathcal{M}} - 1)p_M = c. \end{aligned}$$

Consequently, a row in $\mathcal{N}$ has expected out-degree

$$d_{\mathcal{N}\mathcal{N}} + d_{\mathcal{N}\mathcal{M}} = (n_{\mathcal{N}} - 1 + n_{\mathcal{M}})p_N = (n-1)p_N = c,$$

while a row in $\mathcal{M}$ has expected out-degree $d_{\mathcal{M}\mathcal{M}} = c$. The scalar c controls the total expected row out-degree, and the one-way block structure determines how that degree is distributed across target blocks.

In the balanced design, up to finite-size corrections from the floor operation and the excluded diagonal,

$$d_{\mathcal{N}\mathcal{N}} \approx c/2, \qquad d_{\mathcal{N}\mathcal{M}} \approx c/2, \qquad d_{\mathcal{M}\mathcal{M}} = c.$$

These block-level degrees define two unit-degree crossings in the balanced design. Since $d_{\mathcal{M}\mathcal{M}} = c$, the $\mathcal{M}\mathcal{M}$ expected internal degree crosses one at $c = 1$. Since $d_{\mathcal{N}\mathcal{N}} \approx c/2$, the corresponding $\mathcal{N}\mathcal{N}$ crossing occurs near $c = 2$. We treat neighborhoods of these crossings as candidate effective-degree windows.

The c-grid is refined around $c = 1$ and $c = 2$, the two internal effective-degree crossings induced by the balanced design. These regions are treated as descriptive hypotheses for where longer-lived significant increments may emerge before additional connectivity shifts discoveries toward earlier propagation orders. The grid extends to $c = 50$ to describe the subsequent high-connectivity branch, while the n-grid spans 100 to 10000 to examine how the window–valley separation changes with graph size. The main sweep uses

$$n \in \{100, 250, 500, 1000, 2500, 5000, 10000\}$$

and

$$c \in \left\{ \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1, \frac{9}{8}, \frac{5}{4}, \frac{3}{2}, \frac{7}{4}, 2, \frac{9}{4}, \frac{5}{2}, \frac{11}{4}, 3, 4, 6, 8, 10, 25, 50 \right\}.$$

Each admissible (n, c) cell is replicated 20 times. Admissibility requires $p_N \leq 1$ and $p_M \leq 1$. In the present grid, the only excluded cell is $(n, c) = (100, 50)$, for which $p_M > 1$. No Bernoulli probability is truncated. The design contains 132 admissible cells and 2640 stopping-depth observations.

Under this binary design and threshold $\epsilon = 0.5$, the empirical response is

$$L(n, c) := \eta_0.$$

Each sampled instance is propagated until its significant-increment count satisfies $q_r = 0$; the experiment records the last order with $q_r > 0$.

The regime intervals summarized in Table III are defined from the block-level effective degrees induced by the balanced generator. Since $d_{\mathcal{M}\mathcal{M}} = c$ and $d_{\mathcal{N}\mathcal{N}} \approx c/2$, the two internal effective-degree crossings occur near $c = 1$ for the $\mathcal{M}\mathcal{M}$ block and near $c = 2$ for the $\mathcal{N}\mathcal{N}$ block. We therefore organize the sampled c-grid into a low-connectivity branch below $c = 1$, an $\mathcal{M}$-block window for $1 \leq c \leq 5/4$, an inter-threshold regime for $5/4 < c < 2$, an $\mathcal{N}$-block window for $2 \leq c \leq 3$, and a shortcut-dominated branch above $c = 3$.

Here, a block is said to gain *shortcuts* when its effective degree makes multiple short admissible paths likely between ordered pairs. We use *shortcut-dominated* descriptively for the high-c branch, where such alternatives are expected to shift significant discoveries toward earlier propagation orders and thereby shorten the stopping depth. These regime labels are descriptive summaries of the sampled finite design.

TABLE III
STOPPING-DEPTH SUMMARY BY EMPIRICAL REGIME. MEAN AND $Q_{.95}$ ARE COMPUTED OVER ALL POOLED OBSERVATIONS IN EACH REGIME; MAX. IS THE LARGEST SINGLE OBSERVATION.

Regime	Regime interval	Cells	Mean ($Q_{.95}$)	Max.
Low-connectivity	$c < 1$	21	4.97 (10.0)	13
$\mathcal{M}$-block window	$1 \leq c \leq 5/4$	21	34.87 (79.0)	101
Inter-threshold	$5/4 < c < 2$	14	28.99 (49.0)	57
$\mathcal{N}$-block window	$2 \leq c \leq 3$	35	33.64 (70.0)	102
Shortcut-dominated	$c > 3$	41	10.38 (25.0)	34

VI. EMPIRICAL REGIME ANALYSIS

Fig. 1 reports the empirical stopping depth $L(n, c) = \eta_0$ by sampled n and c. The profiles are finite-range and non-monotone: stopping depth is low in the low-connectivity branch, rises near the $\mathcal{M}$-block window, falls in the inter-threshold regime, rises again near the $\mathcal{N}$-block window for several graph sizes, and declines in the shortcut-dominated branch. The following subsections quantify this pattern by regime, examine its size sensitivity, and use order-level diagnostics to relate the window maxima to the distribution of discoveries across propagation orders.

A. Regime-Level Summary

Table III summarizes $L(n, c) = \eta_0$ by the block-level regimes defined in Section V-B. The regime organization is defined from the effective degrees induced by the generator.

The table quantifies the pattern visible in Fig. 1. The low-connectivity branch has mean $L = 4.97$. The two largest regime-level means occur in the $\mathcal{M}$-block window, with mean $L = 34.87$, and in the $\mathcal{N}$-block window, with mean $L = 33.64$. The inter-threshold regime remains elevated but lower than both windows, with mean $L = 28.99$. These pooled means are descriptive summaries across all sampled sizes; Table IV reports the size-specific contrasts, whose sign and magnitude vary with n.

To quantify the size dependence of the two-window profile, let $\bar{L}_{\mathcal{M}}(n)$, $\bar{L}_{\mathcal{N}}(n)$, and $\bar{L}_I(n)$ denote the mean stopping depths at fixed n in the $\mathcal{M}$-block window, the $\mathcal{N}$-block window, and the inter-threshold regime, respectively. We define

$$\Delta_{\mathcal{M}}(n) = \bar{L}_{\mathcal{M}}(n) - \bar{L}_I(n),$$
$$\Delta_{\mathcal{N}}(n) = \bar{L}_{\mathcal{N}}(n) - \bar{L}_I(n).$$

Here, each mean is computed over the corresponding regime cells at fixed n. Table IV reports these contrasts with bootstrap 95% intervals.

The window–valley separation is size-dependent. For $n \leq 500$, the window–valley contrasts are negative or have intervals that do not establish a positive separation. Near $n = 1000$, both contrasts move toward zero. From $n = 2500$ onward, both bootstrap intervals are strictly positive, and the contrasts

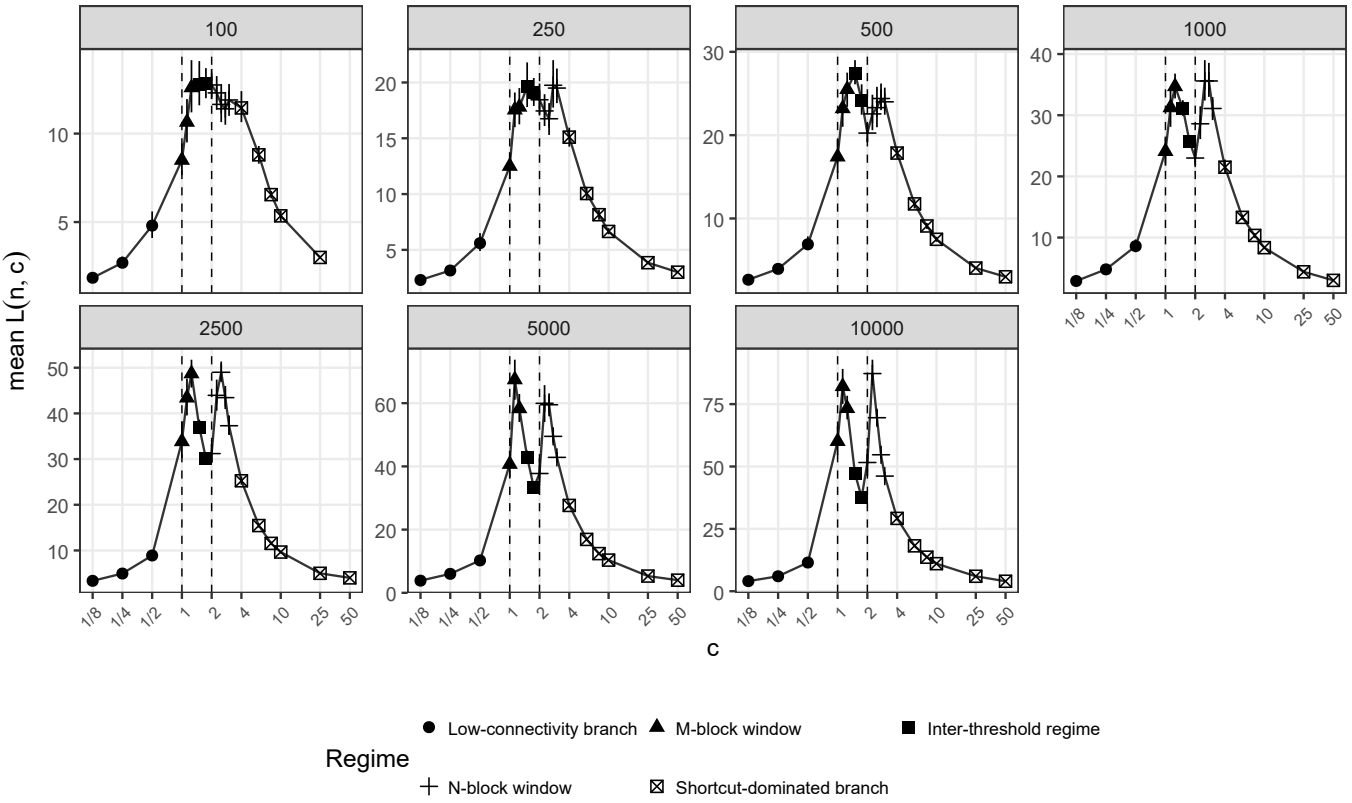


Fig. 1. Fixed-n stopping-depth profiles for the one-way block Bernoulli design. Points report mean $L(n, c) = \eta_0$; error bars report bootstrap 95% intervals for the cell means. Vertical dashed lines mark the effective-degree crossings $c = 1$ and $c = 2$; in the resolved panels, the window maxima occur at sampled c values immediately above these crossings.

TABLE IV
WINDOW–VALLEY CONTRASTS BY GRAPH SIZE. THE CONTRASTS COMPARE EACH WINDOW MEAN WITH THE INTER-THRESHOLD MEAN AT FIXED n. INTERVALS ARE BOOTSTRAP 95% INTERVALS.

n	$\Delta_{\mathcal{M}}(n)$	$\Delta_{\mathcal{N}}(n)$
100	-2.8 $[-4.1, -1.7]$	-1.4 $[-2.5, -0.4]$
250	-4.6 $[-5.9, -3.3]$	-2.2 $[-3.4, -1.1]$
500	-2.5 $[-4.6, -0.5]$	-1.1 $[-2.9, 0.5]$
1000	0.8 $[-1.6, 3.2]$	1.6 $[-0.3, 3.5]$
2500	8.9 $[6.0, 12.0]$	7.5 $[5.0, 9.9]$
5000	17.4 $[12.8, 22.1]$	12.8 $[9.8, 15.8]$
10000	23.9 $[19.2, 28.4]$	15.4 $[11.9, 18.8]$

increase with graph size. At $n = 10000$, the contrasts reach 23.9 and 15.4 for the $\mathcal{M}$- and $\mathcal{N}$-window comparisons, respectively. The two-window profile is therefore a large-size feature of the sampled design that sharpens with graph size.

This empirical ordering is consistent with the effective-degree interpretation used to define the regimes. Below the first crossing, both internal block mechanisms remain sparse and significant increments are exhausted after few orders. Near the $\mathcal{M}$-block window, $d_{\mathcal{MM}} = c$ is close to its effective-degree crossing while $d_{\mathcal{NN}}$ remains below its own window. Between the two crossings, the $\mathcal{MM}$ block has already accumulated shorter alternatives, while the $\mathcal{NN}$ block has not yet reached its corresponding window. Near the $\mathcal{N}$-block window, $d_{\mathcal{NN}} \approx c/2$ approaches its effective-degree crossing. Beyond both windows, shortcuts concentrate significant discoveries at earlier propagation orders.

B. Size-Sensitivity Diagnostic

As a secondary diagnostic, we estimate the fixed-c slope of $L(n, c)$ with respect to $\log n$. This quantity summarizes how strongly the observed stopping depth changes with graph size at each sampled connectivity level. We denote this descriptive slope by $\widehat{\Theta}(c)$.

The diagnostic should be read together with the regime-level profiles. It is not used as the primary explanatory model, because the one-way block generator makes the scalar c correspond to different effective degrees across blocks.

Let $L_b(n, c)$ denote the observed stopping depth in replica b. For each fixed c, the size-sensitivity diagnostic fits $L_b(n, c) = a(c) + \widehat{\Theta}(c) \log n + \xi_b(n, c)$ over the sampled graph sizes. Bootstrap intervals are obtained by resampling replicas within each (n, c) design cell.

The diagnostic is itself two-windowed. It attains local maxima in the two windows, $\widehat{\Theta}(9/8) = 15.15$ and $\widehat{\Theta}(9/4) = 13.80$, separated by a dip to $\widehat{\Theta}(7/4) = 5.34$ in the inter-threshold regime. The corresponding 95% intervals, $[14.02, 16.27]$ and $[12.78, 14.82]$, overlap, so this diagnostic

TABLE V
ORDER-LEVEL DISCOVERY DIAGNOSTICS AT THE FIRST STABILIZED PANEL, $n = 2500$. THE COLUMN D REPORTS $\sum_r q_r$, AVERAGED OVER REPLICAS AND SCALED BY 10^3.

c	D (10^3)	$\bar{r}_q$	Mean L
1	46.7	10.93	35.65
9/8	177.8	15.72	46.05
5/4	445.4	16.35	46.30
3/2	1083.0	13.36	38.15
2	2004.9	9.34	32.70
9/4	2349.1	8.55	43.95
5/2	2718.1	8.29	49.00
11/4	3055.9	8.10	43.45
3	3313.4	7.61	36.95

supports a two-window pattern without ranking one window above the other. It then decays through the shortcut-dominated branch, with $\widehat{\Theta}(10) = 1.28$ and $\widehat{\Theta}(50) = 0.35$.

C. Order-Level Discovery Diagnostic

The order-level counts q_r indicate where discoveries occur across propagation orders. In the binary design with $\epsilon = 0.5$, each ordered pair can contribute at most one significant increment; hence q_r counts first significant discoveries at order r. For each sampled instance, this profile determines three related but distinct quantities:

$$D = \sum_r q_r,$$
$$\bar{r}_q = \frac{\sum_r r q_r}{D},$$
$$L = \eta_0 = \max\{r : q_r > 0\}.$$

The aggregate count D measures how many ordered pairs are discovered during propagation. The weighted mean $\bar{r}_q$ measures where the discovery mass is centered across propagation orders. The stopping depth L, by contrast, is a right-edge statistic: it records the last propagation order at which at least one discovery remains.

We report the order-level diagnostic at $n = 2500$ because Table IV shows that this is the smallest sampled size for which both window–valley bootstrap intervals are strictly positive. It is therefore the first stabilized panel in the sampled n-grid. The regime summaries and scaling analysis use the full sampled design; Table V provides a mechanistic view of the discovery-order distribution at the first stabilized panel.

This table separates aggregate discovery volume from the location of the window maxima. In the first window, D increases from 46.7×10^3 at $c = 1$ to 445.4×10^3 at $c = 5/4$, while the mean discovery order also increases from 10.93 to 16.35. The discovery distribution therefore shifts both upward and later, and the mean stopping depth reaches 46.30. At $c = 3/2$, D continues to grow, reaching 1083.0×10^3, but the mean discovery order falls to 13.36. The additional discoveries are then concentrated at earlier propagation orders, and the mean stopping depth declines to 38.15.

The second window shows the same separation between discovery volume and discovery depth. From $c = 2$ to $c = 3$, D increases monotonically from 2004.9×10^3 to 3313.4×10^3. The mean stopping depth, however, peaks earlier, at $c = 5/2$, with $L = 49.00$. Beyond that point, the discovery count still increases, but the mean stopping depth decreases to 43.45 at $c = 11/4$ and 36.95 at $c = 3$. Thus, high-depth cells are characterized by discovery distributions whose right edge remains active after the aggregate discovery count has grown large.

VII. DISCUSSION, LIMITATIONS, AND FUTURE WORK

The computational contribution of **forgethreads** is an execution-level reformulation of the max–min convolution in (1). The recurrence is unchanged, but each update is evaluated directly at the output-cell level instead of through tensor-broadcasted intermediate arrays. The benchmark shows two practical consequences: runtime decreases substantially relative to the TensorFlow-based reference, and working memory remains quadratic in N_{tot}. The latter extends the dense problem sizes that can be processed on a fixed GPU and supports repeated large-instance stopping-depth experiments without cubic intermediate storage.

The stopping-depth experiment shows a finite-range, nonmonotone response to connectivity. Increasing c does not simply make propagation deeper. Within the sampled one-way design, high stopping depths occur near two block-level effective-degree windows, separated by an inter-threshold valley and followed by a shortcut-dominated decline. In balanced blocks, the same scalar c induces $d_{\mathcal{MM}} = c$ and $d_{\mathcal{NN}} \approx d_{\mathcal{NM}} \approx c/2$. The empirical profile is therefore consistent with an interpretation in terms of block-level effective degrees rather than global connectivity alone.

The fixed-c log-size slope $\widehat{\Theta}(c)$ provides a complementary diagnostic. Its two-window shape indicates that size sensitivity is strongest near the same effective-degree regions that structure the fixed-n profiles. The order-level counts q_r further show that stopping depth is a right-edge property of the discovery-order distribution: larger discovery pools need not produce larger η_0 when additional connectivity shifts discoveries toward earlier propagation orders. In computational terms, these windows identify the sampled regions that require the largest number of propagation iterations.

The binary Bernoulli design should be interpreted as a control design for the stopping criterion. It separates support-level propagation from variability introduced by edge weights and makes nonzero increments unambiguous at $\epsilon = 0.5$. This specialization does not exhaust the weighted forgotten-effects setting. For weighted fuzzy matrices under the same max–min recurrence, threshold crossings also depend on the edge-weight distribution. Because the max–min recurrence uses the idempotent triangular norm $\min$ [5], a path value is determined by its weakest edge rather than by a compounding length penalty. More general strict triangular norms introduce length-dependent attenuation when edge weights are below one. Extending the regime analysis to a max-T framework would therefore require accounting jointly for support structure, edge-weight distributions, and the path-value attenuation induced by the selected composition operator.

The empirical regime map is also limited to the sampled finite design. It covers $n \in \{100, 250, 500, 1000, 2500, 5000, 10000\}$, balanced blocks, and the admissible c-grid defined in Section V-B. Additional experiments with non-balanced blocks, threshold-equalized effective degrees, and larger graph sizes would help distinguish finite-size behavior from more stable regime structure. The present diagnostics are also aggregate: they record q_r but not classwise stopping depths such as $\eta_0^{\mathcal{N}\mathcal{N}}$, $\eta_0^{\mathcal{N}\mathcal{M}}$, and $\eta_0^{\mathcal{M}\mathcal{M}}$, nor the ordered-pair class attaining the extremal depth in each replica. Recording these quantities would allow the two-window profile to be attributed more directly to $\mathcal{M}$-internal, $\mathcal{N}$-internal, and $\mathcal{N} \to \mathcal{M}$ propagation.

Finally, the current implementation uses dense quadratic buffers. This design is appropriate for the dense CUDA comparison reported here, but substantially larger sparse instances would benefit from adjacency-list or CSR-based representations. Developing sparse GPU kernels for cumulative max–min propagation is therefore the main computational extension of **forgethreads**.

VIII. Conclusion

This paper presented **forgethreads**, a CUDA implementation that avoids cubic intermediate tensors through output-cell max–min propagation. On the tested RTX 4090, it obtained $12.8\times$–$44\times$ speedups over the half-precision TensorFlow reference and completed dense benchmark instances up to $32{,}000$ nodes.

Using the same iterative propagation model, we examined η_0, the last order with $q_r > 0$, in 2640 binary one-way block Bernoulli instances. Within the sampled finite design, high stopping depths appear near two induced effective-degree windows, separated by an inter-threshold valley and followed by a shortcut-dominated decline. This finite-range profile is consistent with organization by block-level effective degrees rather than by global connectivity alone. The q_r diagnostics further show that larger discovery pools need not imply larger stopping depths when discoveries shift toward earlier orders.

A natural theoretical continuation is to organize these observations into a regime description of stopping depth indexed by block-level effective degrees. At the threshold $\epsilon = 0.5$ used here, the uniqueness of significant increments provides a deterministic anchor for such an analysis. The present experiment delimits the support-level behavior that this theory must reproduce.

Acknowledgment

The authors acknowledge the Department of Computer Science at Universidad Católica de Temuco for providing access to computational resources on its GPU cluster.

References

[1] A. Kaufmann and J. Gil-Aluja, *Models for the Research of Forgotten Effects*. Milladoiro, 1988.

[2] F. Chávez-Bustamante, E. Mardones-Arias, J. Rojas-Mora, and J. Tijmes-Ihl, "A forgotten effects approach to the analysis of complex economic systems: Identifying indirect effects on trade networks," *Mathematics*, vol. 11, no. 3, p. Art. no. 531, 2023.

[3] E. Sanchez, "Resolution of composite fuzzy relation equations," *Information and Control*, vol. 30, no. 1, pp. 38–48, 1976.

[4] A. Di Nola, S. Sessa, W. Pedrycz, and E. Sanchez, *Fuzzy Relation Equations and Their Applications to Knowledge Engineering*. Dordrecht, The Netherlands: Kluwer Academic Publishers, 1989.

[5] E. P. Klement, R. Mesiar, and E. Pap, *Triangular Norms*. Dordrecht, The Netherlands: Kluwer Academic Publishers, 2000.

[6] H.-J. Zimmermann, *Fuzzy Set Theory and Its Applications*. Springer, 2001.

[7] NVIDIA Corporation, *CUDA C++ Programming Guide*, NVIDIA Corporation, accessed: 2026-06-26. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[8] P. Harish and P. J. Narayanan, "Accelerating large graph algorithms on the GPU using CUDA," in *High Performance Computing – HiPC 2007*, ser. Lecture Notes in Computer Science, vol. 4873. Berlin, Germany: Springer, 2007, pp. 197–208.

[9] A. Buluç, J. R. Gilbert, and C. Budak, "Solving path problems on the GPU," *Parallel Computing*, vol. 36, no. 5–6, pp. 241–253, 2010.

[10] N. Li, X. Yu, and W. Liu, "Max–min transitive closure of randomly generated fuzzy matrix: Bernoulli and classical probabilistic models," *Axioms*, vol. 14, no. 9, p. 690, 2025.

[11] W. Jakob, J. Rhinelander, and D. Moldovan, "pybind11: Seamless operability between C++11 and Python," Software, 2017, accessed: 2026-06-26. [Online]. Available: https://github.com/pybind/pybind11

[12] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," Google Research, Tech. Rep., 2015. [Online]. Available: https://www.tensorflow.org/

[13] C. Araya-Toro and J. Rojas-Mora, "forgeffects: A python package for analyzing forgotten and direct effects," Software, version 0.2.x, 2024, accessed: 2026-06-26. [Online]. Available: https://github.com/claudio-araya/forgeffects

[14] C. Riquelme Ávila, "feCUDA: Finding extremal CUDA paths," GitHub repository, 2026, accessed: 2026-06-26. [Online]. Available: https://github.com/criquelme2003/feCUDA